

From Feedback Loops to Policy Updates: Reinforcement Fine-Tuning for LLM-Based Alpha Factor Discovery

Lingzhe Zhang, Tong Jia*, Yunpeng Zhai, Zixuan Xie, Chiming Duan,
Minghua He, Philip S. Yu, *Fellow, IEEE* and Ying Li*, *Member, IEEE*

Abstract—Modern quantitative trading increasingly relies on systematic models to extract predictive signals from large-scale financial data, where alpha factor discovery plays a central role in transforming market observations into tradable signals. Recent LLM-based methods have shown promise in automating factor generation, but most of them still rely on prompt-level generation–evaluation–feedback loops for iterative optimization. As the loop becomes longer, repeatedly appended historical candidates and feedback can cause context explosion, increase inference cost, dilute useful information, and introduce feedback drift. Moreover, these methods often depend on very large LLMs whose stable generation preferences may lead to structurally similar expressions, redundant candidates, and search stagnation. To address these limitations, we propose QUANTEVOLVER, a self-evolving alpha factor discovery framework based on reinforcement fine-tuning. Instead of accumulating feedback in the prompt, QUANTEVOLVER converts executable quantitative evaluation into policy updates, enabling a Miner LLM to internalize historical optimization experience through parameter learning. Specifically, QUANTEVOLVER constructs high-quality seed factors, builds diverse seed–time-window training tasks, generates executable Factor DSL expressions, evaluates them through Regime Backtest, and optimizes the Miner LLM with Diversity-Complementarity Reward. During training, high-quality factors are continuously accumulated in a Mined Factor Database, which serves as the final discovered factor library. Extensive experiments on three realistic market benchmarks demonstrate the effectiveness of QUANTEVOLVER, which consistently improves the primary evaluation metric of each task over existing LLM-based alpha factor discovery baselines, produces higher-quality and more complementary factor pools.

Index Terms—Quantitative Factor Discovery, Reinforcement Fine-Tuning, Large Language Model

I. INTRODUCTION

Modern quantitative trading increasingly relies on systematic models to extract predictive signals from large-scale financial data. Unlike many conventional prediction settings, financial markets are highly noisy, weakly structured, and non-stationary: useful signals are often sparse, unstable across regimes, and easily overwhelmed by spurious correlations.

Lingzhe Zhang, Tong Jia, Chiming Duan, Minghua He and Ying Li are with Peking University, Beijing, China. Yunpeng Zhai is with Alibaba Group, China. Zixuan Xie are with Nanjing University, Nanjing, China. Philip S. Yu are with University of Illinois Chicago, United States.

Email: {zhang.lingzhe, duanchiming, hemh2120}@stu.pku.edu.cn, {jia.tong, li.ying}@pku.edu.cn, zhaiyunpeng.zyp@alibaba-inc.com and psyu@uic.edu

* Corresponding author: Tong Jia, e-mail: (jia.tong@pku.edu.cn); Ying Li, e-mail: (li.ying@pku.edu.cn)

As a result, the central challenge in quantitative research is not merely to fit historical observations, but to continuously discover robust and economically meaningful signals that can generalize across assets, time periods, and market conditions.

Alpha factors provide a fundamental abstraction for this challenge. An alpha factor is a numerical signal designed to capture predictive patterns related to future returns, risks, or cross-sectional asset rankings. However, alpha factor discovery remains a challenging search problem. First, the candidate space is combinatorially large, as factors can be composed from diverse market variables, mathematical operators, temporal windows, and normalization schemes. Second, financial signals are typically weak, noisy, and regime-dependent, making it difficult to distinguish genuinely predictive factors from spurious historical correlations. Third, useful factors should not only achieve strong empirical performance, but also remain robust across time periods and market conditions, while providing non-redundant information beyond existing signals. These challenges make the scalable and reliable discovery of alpha factors a central problem in quantitative finance.

To address the difficulty of alpha factor discovery, extensive studies have explored automated factor mining beyond manual factor engineering. Existing non-LLM methods generally fall into two categories. The first category adopts rule-based symbolic search, such as genetic programming, symbolic regression, and operator-template enumeration [1]–[6]. These methods represent alpha factors as symbolic expressions and explore the candidate space through predefined operators, mutation rules, crossover operations, or syntactic constraints. The second category introduces learning-guided generation, where trainable models or reinforcement learning policies are used to construct, rank, or select factor expressions based on historical evaluations or previously discovered factors [5], [7]–[11]. Although these methods reduce manual engineering effort, their search processes are still largely shaped by predefined operators, action spaces, expression templates, or task-specific objectives. As a result, they often struggle to flexibly compose factor expressions from high-level market semantics, scenario descriptions, and prior research hypotheses.

Large language models (LLMs) offer a promising way to alleviate these limitations. With their capabilities in symbolic reasoning, code-like generation, and instruction following, LLMs can generate and refine alpha expressions from alpha libraries, market contexts, human instructions, and empirical feedback. Existing LLM-based frameworks have therefore

evolved from interactive factor generation to feedback-driven and search-augmented alpha mining. Early systems, such as Alpha-GPT [12] and FAMA [13], use LLMs as factor miners or interactive assistants. Later frameworks, such as QuantAgent [14], AlphaBench [15], and AlphaAgent [16], further involve LLMs in factor evaluation and refinement by incorporating historical experiment records, human knowledge, or market insights. More recent search-augmented and agentic methods, including LLM-powered MCTS [17], R&D-Agent [18], and QuantaAlpha [19], combine LLM generation with backtesting feedback, multi-agent workflows, or evolutionary exploration.

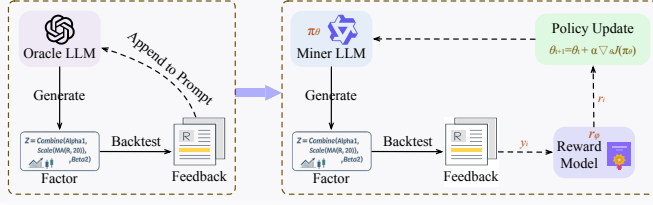


Fig. 1. LLM-Based Alpha Factor Discovery: From Feedback Loops to Policy Updates.

Although these LLM-based methods have demonstrated promising results in generating effective alpha factors, they still largely rely on prompt-level feedback loops for iterative optimization, as illustrated in Figure 1. This design is flexible and easy to implement, but becomes increasingly inefficient and unstable when scaled to large factor mining campaigns, leading to two key challenges:

- **Context explosion and feedback drift.** Existing LLM-based factor mining methods often rely on multi-round generation–evaluation–feedback loops, where historical candidates, evaluation results, and feedback signals are repeatedly appended to the prompt. As the loop becomes longer, the accumulated context increases inference cost, dilutes useful feedback, and introduces irrelevant or outdated information. This makes the feedback-driven optimization process increasingly unstable and may gradually drift away from the original factor mining objective.
- **Large-model dependence and search stagnation.** Existing methods typically depend on very large LLMs to generate syntactically valid and semantically meaningful factor expressions. However, such models often exhibit stable generation preferences inherited from pretraining and prompting, repeatedly producing structurally similar expressions or reusing familiar operator patterns. This limits exploration diversity, causes redundant factor candidates, and weakens the ability to discover genuinely novel alpha signals at scale.

To fill these gaps, we propose QUANTEVOLVER, a self-evolving alpha factor discovery framework based on reinforcement fine-tuning.

To address **Challenge 1**, QUANTEVOLVER replaces prompt-level feedback accumulation with reinforcement fine-tuning. As illustrated in Figure 1, instead of appending long histories of generated factors, evaluation results, and refinement feedback to the prompt, each training round only uses bounded generation–evaluation feedback. The optimization experience

accumulated across rounds is then internalized into the policy model through parameter updates. In this way, QUANTEVOLVER uses implicit optimization memory to replace explicit long-context feedback histories, reducing context growth and mitigating feedback drift during large-scale factor mining.

To address **Challenge 2**, QUANTEVOLVER adopts a lightweight LLM (parameter $\leq 30B$) as the base policy and continuously evolves its generation distribution through executable quantitative rewards. Each generated DSL factor expression is compiled and evaluated on historical market data, and the resulting reward is used to update the policy toward more effective factor proposals. Moreover, we design *DiCo Reward*, a *Diversity-Complementarity* reward mechanism that calibrates raw predictive performance with novelty, redundancy, behavioral diversity, and complementarity signals. This reward design encourages the policy to generate factors that are not only predictive, but also structurally diverse, behaviorally distinct, and complementary to existing candidates.

Beyond these two designs, QUANTEVOLVER further introduces a seeded task construction strategy to stabilize reinforcement fine-tuning. Specifically, an oracle LLM is first used to construct a high-quality seed pool by generating candidate factor expressions, checking DSL validity, empirically scoring candidates, and selecting elite non-redundant seeds. The selected seeds are then expanded by a task bank builder into regime-aware training tasks, where each task contains a seed expression, time split, factor family, mutation hints, and scenario metadata. This design provides meaningful starting points and structured training contexts, enabling the lightweight LLM to focus on improving and diversifying promising factor candidates rather than searching from scratch.

We conduct experiments on three realistic market benchmarks spanning short-horizon single-asset direction prediction, high-frequency cross-sectional factor mining, and daily equity ETF factor discovery. The results show that QUANTEVOLVER consistently improves the primary evaluation metric of each task over existing LLM-based alpha factor discovery methods. Specifically, QUANTEVOLVER achieves a 7.8% relative improvement in directional accuracy on the single-asset direction prediction task. On the high-frequency cross-sectional benchmark, it exceeds the strongest baseline by 109.5% in best out-of-sample RankIC and by 186.9% in top-10 out-of-sample RankIC mean. On the daily equity ETF benchmark, QUANTEVOLVER further produces a high-quality and fuseable factor pool, achieving strong out-of-sample RankIC under a conventional equity-market evaluation protocol. These results demonstrate that QUANTEVOLVER is effective across heterogeneous factor discovery settings, from high-frequency digital-asset prediction to daily equity factor mining. In summary, the key contributions of this work are as follows:

- We introduce a policy-update-driven paradigm for LLM-based alpha factor discovery. Specifically, we replace prompt-level feedback accumulation with reinforcement fine-tuning, allowing a small language model to internalize historical optimization experience through parameter updates and progressively improve its factor generation policy.
- Built upon the above paradigm, we propose QUANTEVOLVER, a self-evolving alpha factor discovery framework.

QUANTEVOLVER combines oracle-based seed construction, regime-aware task bank building, executable quantitative evaluation, and DiCo Reward to provide stable training tasks and reward signals that encourage predictive, diverse, and complementary alpha factors.

- We validate the effectiveness of QUANTEVOLVER on three realistic market benchmarks. Extensive experiments show that QUANTEVOLVER surpasses state-of-the-art LLM-based methods on the primary evaluation metrics, especially in terms of factor quality, diversity, and out-of-sample effectiveness.

II. BACKGROUND

In this section, we present the essential background of this paper, including the formal definition of alpha factor discovery and the emerging use of reinforcement fine-tuning (RFT) for adapting LLMs to task-specific feedback.

A. Alpha Factor Discovery

Alpha factors are numerical signals designed to capture predictive patterns in financial markets. Given historical market observations, such as prices, volumes, returns, and derived technical indicators, an alpha factor maps these observations into a scalar signal that can be used for forecasting future returns, ranking assets, or constructing trading portfolios. At a fundamental level, an alpha factor can be viewed as a function from market observations to numerical signals. Therefore, many alpha factors can be naturally expressed in a formulaic form, where market variables are composed through mathematical operators, time-series transformations, and cross-sectional operations.

For example, a simple momentum-style factor may compute the recent return trend of an asset, while a reversal-style factor may rank assets according to short-term price declines. A formulaic alpha factor can be expressed as a symbolic expression, such as Equation 1.

$$f = \text{rank}(\text{ts_mean}(\text{return}, 5)), \quad (1)$$

Here, `ts_mean` computes a moving average over a five-period window and `rank` normalizes the resulting values across assets. Such expressions are compact and executable: once the variables and operators are defined, the expression can be directly applied to historical market data to produce factor values.

Formally, let $\mathcal{X}$ denote historical market data and let $\mathcal{L}$ denote a domain-specific language (DSL) that defines the valid variables, operators, and composition rules for constructing alpha expressions. An alpha factor can be represented as an expression $f \in \mathcal{L}$, which maps market data $\mathcal{X}$ to a factor value sequence as Equation 2.

$$z_f = f(\mathcal{X}). \quad (2)$$

Here, the output z_f denotes the factor values produced by applying expression f to market data $\mathcal{X}$. Given a target market outcome y , such as future return or cross-sectional asset ranking, the empirical quality of f can be measured by

an evaluation function $M(z_f, y)$. Common evaluation signals include predictive accuracy, information coefficient (IC), rank information coefficient (RankIC), and portfolio-level performance indicators.

In practical quantitative pipelines, alpha factors are rarely used in isolation. Instead, multiple factors are usually combined into a factor set and further fused into a composite signal for prediction, ranking, or portfolio construction. Given a discovered factor set as shown in Equation 3.

$$\mathcal{F} = \{f_1, f_2, \dots, f_n\}, \quad f_i \in \mathcal{L}, \quad (3)$$

The corresponding factor values can be denoted as Equation 4.

$$\mathbf{Z}_{\mathcal{F}} = [z_{f_1}, z_{f_2}, \dots, z_{f_n}], \quad (4)$$

Here, each z_{f_i} is computed according to Equation 2. A downstream fusion model or aggregation function g can then combine these factor values into a final predictive signal as Equation 5.

$$s = g(\mathbf{Z}_{\mathcal{F}}). \quad (5)$$

It illustrates why factor discovery is not only about finding individually strong factors. If newly discovered factors are highly redundant, they may provide limited incremental value after fusion. Therefore, an effective factor set should contain factors that are predictive, diverse, and complementary to each other.

The goal of alpha factor discovery is to search for a set of factor expressions $\mathcal{F}$ that achieves strong empirical performance under validation protocols and provides useful signals for downstream fusion. This problem is challenging because the candidate space grows rapidly with the number of variables, operators, temporal windows, and composition patterns. Moreover, financial signals are noisy and regime-dependent, so factors that appear effective in one period may fail to generalize in another. In practice, discovered factors are also expected to provide incremental value beyond existing factor libraries, since highly similar expressions often contribute limited new information.

B. Reinforcement Fine-Tuning

Reinforcement fine-tuning (RFT) is a training paradigm that adapts language models using task-specific reward signals. Unlike supervised fine-tuning (SFT), which learns from reference outputs or demonstrations, RFT allows a model to generate candidate outputs, receive rewards from an external evaluator, and update its policy according to the evaluated quality of these outputs. This makes RFT suitable for structured generation tasks where high-quality outputs are difficult to exhaustively annotate but can be assessed through task-specific feedback, such as mathematical reasoning, code generation, and tool-use planning [20], [21].

Formally, given an input prompt x , a language model policy π_{θ} generates an output $y \sim \pi_{\theta}(\cdot | x)$. A reward function $R(x, y)$ evaluates the quality of the generated output. The

objective of RFT is to update the policy parameters θ to maximize the expected reward as Equation 6.

$$\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot | x)} [R(x, y)]. \quad (6)$$

Here, RFT converts task feedback into parameter-level policy updates. Through repeated optimization, the model can gradually internalize task-specific preferences and improve its generation behavior, rather than relying only on in-context examples or manually written feedback prompts.

Modern RFT methods differ in how rewards are collected and how policy updates are performed. Proximal Policy Optimization (PPO) [22] is widely used because it stabilizes policy updates through clipped objectives or trust-region-style constraints. Direct Preference Optimization (DPO) [23] simplifies preference optimization by directly learning from preference pairs without explicitly training a separate reward model. More recently, Group Relative Policy Optimization (GRPO) [24] has been used to improve reasoning-oriented language models by comparing multiple sampled completions under the same prompt. Specifically, for a prompt x , GRPO samples a group of K outputs as Equation 7.

$$\mathcal{Y}(x) = \{y_1, y_2, \dots, y_K\}, \quad y_i \sim \pi_{\theta_{\text{old}}}(\cdot | x). \quad (7)$$

Each output receives a reward $R(x, y_i)$ from a task-specific evaluator. The relative advantage of each output can then be estimated by normalizing rewards within the group as shown in Equation 8.

$$\hat{A}_i = \frac{R(x, y_i) - \text{mean}(\{R(x, y_j)\}_{j=1}^K)}{\text{std}(\{R(x, y_j)\}_{j=1}^K)}. \quad (8)$$

Here, GRPO encourages the model to increase the probability of outputs that outperform other candidates generated under the same prompt, while suppressing relatively poor outputs. This group-wise comparison is useful for structured generation tasks, where multiple candidates may be syntactically valid but differ substantially in task-level quality.

Despite the success of RFT in reasoning, code generation, and other structured generation tasks, its use in LLM-based alpha factor discovery remains underexplored. Existing LLM-based alpha mining methods mainly rely on prompt-level interaction and iterative feedback, while few studies investigate whether reinforcement fine-tuning can directly improve the factor generation policy itself.

III. METHODOLOGY

The workflow of QUANTEVOLVER is illustrated in Figure 2. Given a user-specified factor mining scenario, QUANTEVOLVER first performs seed construction, where the raw scenario is refined into a structured factor scenario and an oracle LLM generates raw seed candidates. These candidates are then validated, scored, and deduplicated to form a high-quality seed pool. By combining the selected seeds with different time splits in the backtesting period, QUANTEVOLVER further constructs a task bank that provides diverse and structured training instances for subsequent reinforcement fine-tuning.

Based on the task bank, QUANTEVOLVER conducts RFT-based factor discovery with a miner LLM. The miner LLM iteratively generates candidate factor expressions, receives executable quantitative feedback, and is updated through in-the-loop GRPO fine-tuning. To guide this process, QUANTEVOLVER introduces DiCo Reward (Diversity-Complementarity Reward), which encourages the model to discover factors with strong predictive quality, diversity, and complementarity. High-quality factors generated throughout training are continuously stored in a mined factor database, and the final output is a validated factor library tailored to the target scenario.

A. Seed Construction

The goal of seed construction is to provide high-quality starting points for reinforcement fine-tuning. Instead of asking the miner LLM to explore the entire factor expression space from scratch, QUANTEVOLVER constructs a compact seed pool from a user-specified factor mining scenario or a prepared scenario configuration. This design improves the stability of subsequent policy optimization and guides the miner LLM toward promising regions of the factor space.

Given a raw user scenario u , QUANTEVOLVER may first perform scenario refinement to convert it into a structured factor scenario as Equation 9, where s contains task-specific information such as the target market, prediction horizon, available variables, factor constraints, and evaluation objective.

$$s = \phi_{\text{refine}}(u), \quad (9)$$

Based on the structured scenario s , a strong LLM or a prepared seed source proposes a set of raw seed candidates as Equation 10, where each c_i is a candidate factor expression written in the predefined factor expression language.

$$\mathcal{C}_{\text{raw}} = \{c_1, c_2, \dots, c_m\} = \phi_{\text{seed}}(s), \quad (10)$$

The raw candidates are then filtered through DSL validation, empirical scoring, and canonical deduplication. First, DSL validation checks whether each candidate expression satisfies the syntax, operator, and type constraints of the factor language. Let $\mathcal{L}$ denote the valid expression space. The valid candidate set is defined as Equation 11.

$$\mathcal{C}_{\text{valid}} = \{c_i \in \mathcal{C}_{\text{raw}} \mid c_i \in \mathcal{L}\}. \quad (11)$$

Second, each valid candidate is executed on historical market data and assigned an empirical score. Given market data $\mathcal{X}$ and an evaluation metric $M(\cdot)$, the score of candidate c_i is computed as Equation 12.

$$q(c_i) = M(c_i(\mathcal{X})). \quad (12)$$

Here, $q(c_i)$ measures the initial empirical quality of the candidate under the target scenario. The specific metric is selected according to the benchmark objective, such as DirAcc for directional prediction or RankIC for cross-sectional ranking.

Third, QUANTEVOLVER removes redundant candidates to avoid constructing a seed pool dominated by duplicated or near-identical expressions. In implementation, candidates are

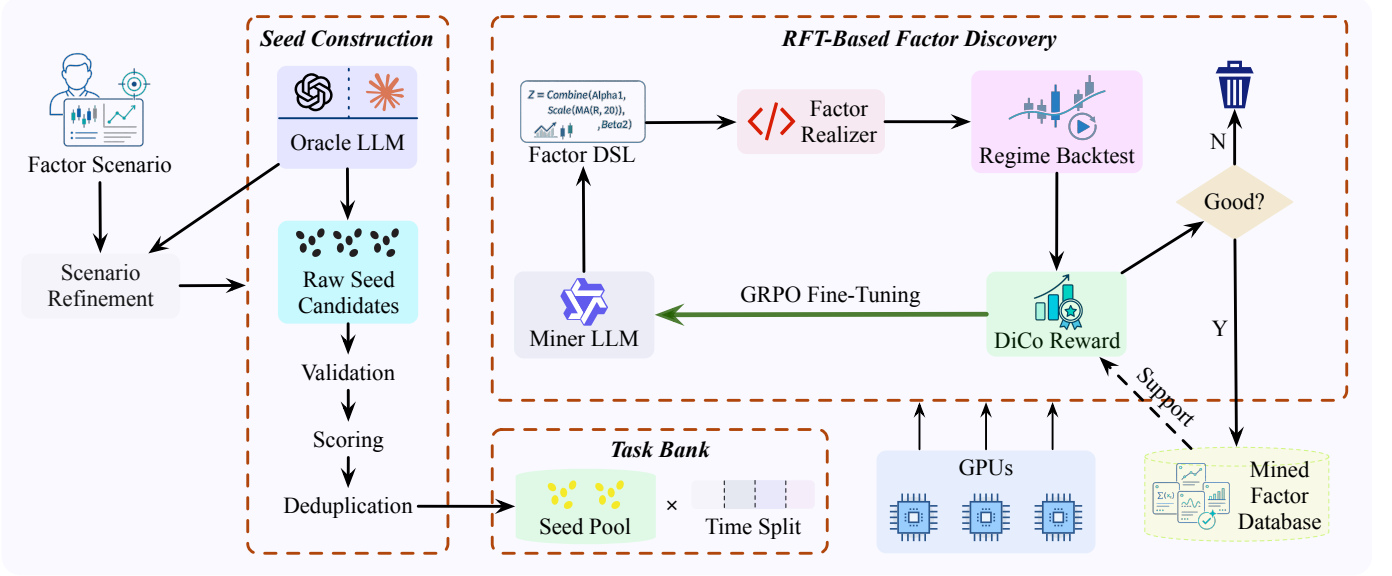


Fig. 2. Overall Framework of QUANTEVOLVER.

ranked by their empirical scores and greedily selected using canonical expression signatures, such as normalized AST hashes. A candidate is retained only if it passes the quality filter and its canonical form has not already been selected as Equation 13, where τ_q is the quality threshold, $h(\cdot)$ denotes the canonical expression signature, and $\mathcal{H}_{\text{selected}}$ is the set of signatures of already selected seeds. The resulting seed pool $\mathcal{S}$ contains valid, effective, and non-redundant factor expressions.

$$\mathcal{S} = \text{TopK}(\{c_i \in \mathcal{C}_{\text{valid}} \mid q(c_i) \geq \tau_q, h(c_i) \notin \mathcal{H}_{\text{selected}}\}) \quad (13)$$

After obtaining the seed pool, QUANTEVOLVER constructs a task bank for reinforcement fine-tuning. Let Equation 14 denote a set of evaluation windows in the backtesting period.

$$\mathcal{W} = \{w_j\}_{j=1}^n = \{(\text{start}_j, \text{end}_j)\}_{j=1}^n \quad (14)$$

Each window specifies the market interval on which generated factors are evaluated during RFT. For each seed $f \in \mathcal{S}$ and each time window $w_j \in \mathcal{W}$, QUANTEVOLVER constructs a seeded factor mining task as Equation 15, where f is the seed expression, s is the structured scenario, w_j is the evaluation window, and o denotes the corresponding factor discovery objective.

$$\tau_{f,j} = (f, s, w_j, o) \quad (15)$$

The full task bank is then defined as the Cartesian product between the seed pool and the evaluation windows as Equation 16.

$$\mathcal{B} = \{\tau_{f,j} \mid f \in \mathcal{S}, w_j \in \mathcal{W}\}. \quad (16)$$

Each task in $\mathcal{B}$ provides a structured training instance for the miner LLM. It specifies which seed expression should be improved, under which market scenario it should be optimized, and on which historical window it should be evaluated. By

combining seeds with multiple evaluation windows, the task bank exposes the miner LLM to diverse market conditions and reduces the risk that reinforcement fine-tuning overfits to a single historical interval. This seeded task construction allows QUANTEVOLVER to focus policy optimization on improving and diversifying promising factor candidates, rather than performing unconstrained exploration from scratch.

B. RFT-Based Factor Discovery

After constructing the task bank, QUANTEVOLVER performs RFT-based factor discovery to optimize the factor generation distribution of the Miner LLM. Different from prompt-level feedback loops that repeatedly append previous candidates and feedback into the context, QUANTEVOLVER uses reinforcement fine-tuning to convert executable quantitative evaluation into parameter updates. As illustrated in Figure 2, the Miner LLM generates candidate expressions in the Factor DSL, the Factor Realizer converts them into executable factor computations, the Regime Backtest module evaluates their empirical performance on historical market windows, and high-quality factors are accumulated in the Mined Factor Database.

Formally, let $\mathcal{B}$ denote the task bank constructed in the previous subsection. Each task $\tau \in \mathcal{B}$ contains a seed expression, a structured market scenario, an evaluation window, and a factor discovery objective. Given a task τ , the Miner LLM, modeled as a policy π_θ , generates a candidate factor expression as Equation 17.

$$\hat{f} \sim \pi_\theta(\cdot \mid \tau). \quad (17)$$

Here, $\hat{f}$ denotes a newly generated or mutated factor expression conditioned on the task information. In the main setting, $\hat{f}$ is required to follow the predefined Factor DSL syntax. For DSL-free ablations, the same framework can instead evaluate generated Python Backtrader factor code.

To make the generated candidate evaluable, QUANTEVOLVER uses the Factor Realizer to parse and convert the Factor DSL expression into executable factor code. Let $\psi_{\text{realize}}(\cdot)$ denote the realization function. The executable implementation of $\hat{f}$ is obtained as Equation 18, where $p_{\hat{f}}$ is executable code that can be applied to historical market data. If the generated output violates the required format, fails Factor DSL validation, contains unsupported operators, or cannot be executed safely, it is treated as invalid and assigned a low reward.

$$p_{\hat{f}} = \psi_{\text{realize}}(\hat{f}), \quad (18)$$

For each valid executable candidate $p_{\hat{f}}$, QUANTEVOLVER performs Regime Backtest on the historical evaluation window specified by τ . Let $\mathcal{X}^\tau$ denote the market data in this window and let y^τ denote the corresponding future-return target. The factor values generated by $p_{\hat{f}}$ are computed as Equation 19.

$$z_{\hat{f}} = p_{\hat{f}}(\mathcal{X}^\tau). \quad (19)$$

The Regime Backtest module then compares the generated factor values against the target outcome as Equation 20, where $\psi_{\text{bt}}(\cdot)$ denotes the task-specific backtesting function and $e_{\hat{f}}$ contains empirical evaluation results, such as DirAcc, IC, RankIC, coverage, validity status, and auxiliary statistics. The primary metric depends on the benchmark objective: for example, DirAcc is used for single-asset directional prediction, while RankIC is used for cross-sectional ranking or single-asset rank correlation tasks.

$$e_{\hat{f}} = \psi_{\text{bt}}(z_{\hat{f}}, y^\tau, \tau), \quad (20)$$

Based on the backtesting result, QUANTEVOLVER computes the training reward as Equation 21, where $\mathcal{A}$ denotes the current Mined Factor Database and auxiliary archives used during training. The detailed design of R_{DiCo} is introduced in the next subsection.

$$r_{\hat{f}} = R_{\text{DiCo}}(\hat{f}, e_{\hat{f}}, \tau, \mathcal{A}), \quad (21)$$

During training, candidates that pass empirical-quality and coverage requirements are inserted into the Mined Factor Database. Let $\eta(\hat{f}, e_{\hat{f}})$ be a selection function that determines whether a generated candidate should be retained as Equation 22.

$$\mathcal{A} \leftarrow \mathcal{A} \cup \left\{ \hat{f} \mid \eta(\hat{f}, e_{\hat{f}}) = 1 \right\}. \quad (22)$$

In implementation, this database-level selection checks whether the candidate is executable, satisfies minimum metric and coverage thresholds, and is not an exact duplicate of an already saved factor. Candidates that do not pass the selection criterion are discarded after their rewards have contributed to policy optimization.

The Miner LLM is updated using group-based reinforcement fine-tuning with GRPO. For each task τ , the Miner LLM samples a group of K candidate outputs as Equation 23.

$$\mathcal{G}_\tau = \{\hat{f}_1, \hat{f}_2, \dots, \hat{f}_K\}, \quad \hat{f}_k \sim \pi_\theta(\cdot \mid \tau). \quad (23)$$

Each candidate $\hat{f}_k$ is realized by the Factor Realizer, evaluated by Regime Backtest, and assigned a reward r_k according to Equation 21. These rewards are then passed to the GRPO optimizer, which performs group-wise policy updates as described in the background section. This process increases the likelihood of candidates that receive higher executable rewards under the same task, while suppressing invalid or low-quality generations.

Overall, RFT-based factor discovery treats reinforcement fine-tuning as a search-distribution optimization mechanism. The Miner LLM is continuously improved through policy updates, but the final output of QUANTEVOLVER is not merely the fine-tuned model. Instead, high-quality factors discovered throughout training are accumulated in the Mined Factor Database, and the final result is a validated factor set tailored to the target scenario.

C. DiCo Reward

DiCo Reward, short for Diversity-Complementarity Reward, is designed to guide RFT-based factor discovery beyond pure predictive performance. A reward based only on empirical score may cause the Miner LLM to repeatedly exploit a small set of similar expressions. Therefore, DiCo Reward augments the task-specific predictive reward with lightweight shaping terms that encourage structural diversity and behavioral complementarity. The required expression-level, family-level, and behavior-level records are maintained in the Mined Factor Database.

Given the generated factor $\hat{f}$ and its Regime Backtest result $e_{\hat{f}}$, DiCo Reward first extracts a task-specific predictive reward $r_{\text{pred}}(\hat{f}) = g(e_{\hat{f}})$, where $g(\cdot)$ extracts the primary metric from the backtesting result. Depending on the benchmark, this metric can be directional accuracy, IC, RankIC, or mean cross-sectional RankIC. Invalid, non-executable, or insufficient-coverage candidates receive a low reward.

To discourage direct repetition, DiCo Reward applies an exact-repeat penalty $r_{\text{exact}}(\hat{f}) = -\lambda_{\text{exact}} \mathbb{I}_{\text{exact}}(\hat{f})$, where $\mathbb{I}_{\text{exact}}(\hat{f})$ indicates whether the normalized expression hash of $\hat{f}$ has already appeared in the Mined Factor Database. This prevents the policy from receiving the same reward for repeatedly generating the same expression.

Beyond exact duplicates, many factors may share the same structural template while differing only in local constants, such as lookback windows. QUANTEVOLVER therefore maintains structural family records in the Mined Factor Database. Let $\rho(\hat{f})$ denote the family signature of $\hat{f}$, and let $N_{\mathcal{A}}(\rho(\hat{f}))$ be the number of previously observed factors from the same family in the Mined Factor Database. The family-level shaping term is written as Equation 24, where $\mathbb{I}_{\text{new}}(\hat{f})$ indicates that $\hat{f}$ is a high-quality factor from a previously unseen structural family, and $\mathbb{I}_{\text{over}}(\hat{f})$ indicates that $\hat{f}$ belongs to an overused low-quality family. This design allows useful local refinement within a family while discouraging excessive exploitation of weak templates.

$$r_{\text{fam}}(\hat{f}) = \lambda_{\text{new}} \mathbb{I}_{\text{new}}(\hat{f}) - \lambda_{\text{fam}} \mathbb{I}_{\text{over}}(\hat{f}), \quad (24)$$

For tasks where behavioral profiles are available, DiCo Reward further introduces a complementarity term. Let $b(\hat{f})$ denote the behavior profile of $\hat{f}$, such as prediction vectors in single-asset timing tasks or cross-sectional rank vectors in RankIC tasks. We define the maximum behavioral similarity to elite factors in the Mined Factor Database as Equation 25.

$$c_{\max}(\hat{f}) = \max_{f \in \mathcal{A}_{\text{elite}}} \text{corr}(b(\hat{f}), b(f)). \quad (25)$$

The complementarity shaping term is then defined as Equation 26, where $\mathbb{I}_{\text{low}}(\hat{f})$ indicates sufficiently low behavioral correlation with elite factors in the Mined Factor Database, and $[x]_+ = \max(x, 0)$. This term rewards behaviorally distinct candidates and penalizes candidates that are highly correlated with already mined elite factors.

$$r_{\text{comp}}(\hat{f}) = \lambda_{\text{low}} \mathbb{I}_{\text{low}}(\hat{f}) - \lambda_{\text{corr}} [c_{\max}(\hat{f}) - \tau_{\text{corr}}]_+. \quad (26)$$

The final reward combines the predictive and shaping components can be calculate as Equation 27.

$$r_{\hat{f}} = \text{clip}(r_{\text{pred}} + r_{\text{exact}} + r_{\text{fam}} + r_{\text{comp}}, r_{\min}, r_{\max}). \quad (27)$$

In practice, the predictive component remains the dominant signal, while the diversity and complementarity terms serve as lightweight regularization. Different tasks may enable different subsets of these components depending on the available behavioral profiles. For example, cross-sectional RankIC tasks can use rank-vector behavior records, while simpler settings may only use exact-repeat and structural-family records.

Overall, DiCo Reward encourages the Miner LLM to discover factors that are not only empirically strong, but also structurally diverse and behaviorally complementary to the existing Mined Factor Database. This helps reduce mode collapse during RFT and improves the quality of the final factor library.

IV. EVALUATION

In this section, we evaluate QUANTEVOLVER from 5 perspectives. overall evaluation, ablation study, hyperparameter sensitivity, training reward analysis, profitability case study.

A. Experimental Setup

Unless otherwise stated, QUANTEVOLVER is instantiated with the Qwen3-14B base model. For downstream evaluation, all methods are assessed under a unified post-selection and fusion protocol: candidate factors are first evaluated individually, then ranked by validation performance, filtered by decorrelation, and finally fused into a multi-factor signal using equal weighting. By default, we adopt validation-guided decorrelated selection with a correlation threshold of 0.7 and report results from the resulting fused portfolio. All experiments are conducted on a local compute server equipped with 160 Intel(R) Xeon(R) CPU cores, 1.8TiB RAM, and 8 NVIDIA H20 GPUs.

B. Metrics

We evaluate the discovered alpha factors from both predictive accuracy and ranking quality. Given a factor f , let $z_{i,t}$ denote the factor value of asset i at time t , and let $r_{i,t+1}$ denote the future return of the same asset in the next period. We use four commonly adopted metrics: directional accuracy (DirAcc), information coefficient (IC), rank information coefficient (RankIC), and information coefficient information ratio (ICIR).

Directional Accuracy (DirAcc). DirAcc measures whether the factor correctly predicts the direction of future returns. It is defined as the proportion of samples where the sign of the factor signal is consistent with the sign of the future return as Equation 28, where Ω denotes the set of valid asset-time samples and $\mathbb{I}[\cdot]$ is the indicator function. A higher DirAcc indicates stronger directional forecasting ability.

$$\text{DirAcc}(f) = \frac{1}{|\Omega|} \sum_{(i,t) \in \Omega} \mathbb{I}[\text{sign}(z_{i,t}) = \text{sign}(r_{i,t+1})], \quad (28)$$

Information Coefficient (IC). IC measures the linear correlation between factor values and future returns. For each time t , we compute the cross-sectional Pearson correlation between $\{z_{i,t}\}_{i=1}^{N_t}$ and $\{r_{i,t+1}\}_{i=1}^{N_t}$ as Equation 29, where N_t is the number of valid assets at time t .

$$\text{IC}_t(f) = \text{corr}_i(z_{i,t}, r_{i,t+1}), \quad (29)$$

The overall IC is then computed as the average over all evaluation periods as Equation 30.

$$\text{IC}(f) = \frac{1}{T} \sum_{t=1}^T \text{IC}_t(f). \quad (30)$$

A higher absolute IC indicates stronger linear association between the factor signal and future returns. In our evaluation, we report IC in the direction where larger values indicate better predictive performance.

Rank Information Coefficient (RankIC). RankIC measures the rank correlation between factor values and future returns. It is computed as the cross-sectional Spearman correlation at each time step as Equation 31 and averaged over all evaluation periods as Equation 32.

$$\text{RankIC}_t(f) = \text{corr}_i(\text{rank}(z_{i,t}), \text{rank}(r_{i,t+1})), \quad (31)$$

$$\text{RankIC}(f) = \frac{1}{T} \sum_{t=1}^T \text{RankIC}_t(f). \quad (32)$$

RankIC is widely used for cross-sectional factor evaluation because it focuses on whether the factor can correctly rank assets by future performance.

Information Coefficient Information Ratio (ICIR). ICIR measures the stability of IC over time. It is defined as the mean IC divided by the standard deviation of IC as Equation 33, where ϵ is a small constant for numerical stability. A higher ICIR indicates that the factor not only has strong

predictive correlation, but also maintains stable performance across different periods.

$$\text{ICIR}(f) = \frac{\text{mean}(\{\text{IC}_t(f)\}_{t=1}^T)}{\text{std}(\{\text{IC}_t(f)\}_{t=1}^T) + \epsilon}, \quad (33)$$

C. Compared Approaches & Benchmark

We compare QUANTEVOLVER with four representative LLM-based alpha factor discovery methods. For a fair comparison, all methods use Qwen-3.6-Plus as the backbone LLM and are evaluated under the same factor expression space, market variables, operator definitions, and validation protocol.

AlphaBench [15] is a systematic benchmark framework for evaluating LLMs in formulaic alpha factor mining. It studies LLM capabilities across factor generation, factor evaluation, and factor searching, and serves as a representative prompt-based LLM alpha mining baseline. :contentReference[oaicite:0]index=0

QuantaAlpha [19] is an evolutionary LLM-driven alpha mining framework. It treats each end-to-end mining run as a trajectory and improves generated factors through trajectory-level mutation and crossover, while enforcing semantic consistency among hypotheses, factor expressions, and executable code. :contentReference[oaicite:1]index=1

R&D-Agent [18] is an LLM-based automated research-and-development framework for data-driven solution building. It adopts a dual-agent design, where a Researcher proposes ideas based on performance feedback and a Developer implements or refines solutions based on execution feedback. We adapt it to alpha factor discovery by treating factor proposal, implementation, and empirical evaluation as an iterative data-driven research workflow. :contentReference[oaicite:2]index=2

Alpha-Jungle [17] refers to the LLM-powered MCTS framework for formulaic alpha factor mining. It combines LLM-based symbolic formula generation with Monte Carlo Tree Search, using quantitative backtesting feedback to guide exploration and frequent-subtree avoidance to improve search efficiency. :contentReference[oaicite:3]index=3

We evaluate all methods on three benchmarks, denoted as **A**, **B**, and **Γ**, which cover single-asset direction prediction, high-frequency cross-sectional factor mining, and daily equity ETF factor discovery.

Benchmark A: real-asset 5-minute directional prediction. Benchmark **A** is constructed from the 5-minute market data of a real liquid asset. At each timestamp, the task is to predict whether the asset price will increase in the next 5-minute interval compared with the current price. Any positive future price change is labeled as an upward movement, while non-positive movement is labeled as non-upward. Since the task is a single-asset binary directional prediction problem, directional accuracy (DirAcc) is used as the primary metric.

Benchmark B: high-frequency cross-sectional factor discovery with hourly rebalancing. Benchmark **B** is constructed from high-frequency multi-asset data collected from a major digital-asset exchange. At each hourly rebalancing timestamp, a discovered factor assigns scores to multiple tradable assets, and the evaluation measures whether these scores can correctly

rank assets by their subsequent returns over the next holding period. This benchmark reflects a realistic high-frequency cross-sectional factor mining setting with hourly portfolio rebalancing, where the method must discover factors that generalize across assets and remain effective under frequent market updates. We use IC, RankIC, and ICIR as evaluation metrics, with RankIC as the primary metric.

Benchmark Γ: daily CSI 300 ETF factor discovery. Benchmark **Γ** is constructed from daily market data of CSI 300 ETF constituents. The task is to discover daily alpha factors that predict future asset returns or cross-sectional rankings under a standard equity factor evaluation protocol. Compared with Benchmarks **A** and **B**, this benchmark represents a lower-frequency and more conventional equity-market factor discovery setting. We evaluate generated factors using IC, RankIC, and ICIR, and use RankIC as the primary metric.

Together, the three benchmarks evaluate alpha factor discovery under complementary settings: Benchmark **A** focuses on short-horizon single-asset direction prediction, Benchmark **B** focuses on high-frequency cross-sectional ranking, and Benchmark **Γ** focuses on daily equity ETF factor discovery.

D. Overall Evaluation

Table I reports the overall performance of QUANTEVOLVER and representative LLM-based alpha factor discovery baselines on the three benchmarks. The highlighted columns indicate the primary metric of each benchmark: DirAcc for Benchmark **A** and RankIC for Benchmarks **B** and **Γ**. Overall, QUANTEVOLVER achieves the best performance on the primary metric across all three benchmarks, demonstrating its effectiveness under heterogeneous factor discovery settings.

On Benchmark **A**, which focuses on short-horizon single-asset directional prediction, QUANTEVOLVER achieves the highest DirAcc of 53.22%, outperforming the strongest baseline R&D-Agent at 52.59%. Although several baselines obtain higher auxiliary IC or ICIR values, these metrics are not the primary objective of this directional prediction task. The improvement in DirAcc indicates that reinforcement fine-tuning helps the Miner LLM discover factors that are better aligned with the target decision rule of next-interval price direction.

On Benchmark **B**, which evaluates high-frequency cross-sectional factor mining, QUANTEVOLVER obtains the strongest RankIC of 0.0586, substantially exceeding the best baseline RankIC of 0.0337. It also achieves the highest IC and ICIR, reaching 0.0500 and 50.2644, respectively. These results show that QUANTEVOLVER not only improves the primary ranking metric, but also produces factors with stronger linear correlation and more stable information coefficients over time. The consistent advantage across IC, RankIC, and ICIR suggests that the discovered factors generalize better across assets in a high-frequency cross-sectional setting.

On Benchmark **Γ**, the daily equity ETF factor discovery benchmark, QUANTEVOLVER achieves the best RankIC of 0.1923 and the best ICIR of 5.4289. It also obtains the highest IC of 0.1502, slightly outperforming AlphaBench. Although AlphaBench achieves the highest DirAcc, RankIC

TABLE I
OVERALL EVALUATION

Method	A				B				T			
	DirAcc*	IC	RankIC	ICIR	DirAcc	IC	RankIC*	ICIR	DirAcc	IC	RankIC*	ICIR
AlphaBench	51.82%	0.0147	0.0362	0.6139	<u>49.57%</u>	<u>0.0288</u>	<u>0.0337</u>	24.4040	55.56%	<u>0.1480</u>	<u>0.1688</u>	<u>2.6021</u>
QuantaAlpha	52.15%	<u>0.0095</u>	0.0313	3.6004	49.41%	0.0216	0.0235	<u>24.4209</u>	<u>54.03%</u>	0.1144	0.1003	2.0239
R&D-Agent	<u>52.59%</u>	0.0002	0.0527	0.9779	49.40%	0.0188	0.0205	19.0101	53.31%	0.1001	0.0589	-0.8667
Alpha-Jungle	52.25%	0.0023	0.0357	<u>1.5664</u>	49.39%	0.0185	0.0200	19.2806	48.92%	0.0369	-0.0008	-0.7166
QUANTEVOLVER	53.22%	-0.0058	<u>0.0481</u>	1.1191	49.96%	0.0500	0.0586	50.2644	53.49%	0.1502	0.1923	5.4289

is the primary metric for this benchmark because the task focuses on factor ranking quality rather than binary direction prediction. The strong RankIC and ICIR results indicate that QUANTEVOLVER can discover daily equity factors with both high predictive ranking quality and stable out-of-sample behavior.

These results support two main observations. First, QUANTEVOLVER consistently improves the primary task metric across all benchmarks, showing that policy-update-driven factor discovery is effective beyond a single market or evaluation protocol. Second, the gains are most pronounced on the two ranking-oriented benchmarks, where the ability to generate diverse and complementary factor candidates is especially important. This confirms that QUANTEVOLVER is not merely optimizing isolated factor expressions, but learning a stronger factor generation policy that transfers across different alpha discovery scenarios.

E. Ablation Study

Table II presents the ablation results on Benchmark B, where RankIC is the primary metric. We study three key components of QUANTEVOLVER: seed construction (**Seed**), diversity-aware reward shaping (**Div**), and the Factor DSL constraint (**DSL**). The full model, which uses all three components, achieves the best performance across all metrics, with an IC of 0.0500, a RankIC of 0.0586, and an ICIR of 50.2644.

TABLE II
ABLATION RESULTS ON DATASET B

Seed	Div	DSL	IC	RankIC*	ICIR
✓	✓	✓	0.0500	0.0586	50.2644
✓		✓	0.0421	0.0505	45.8038
	✓	✓	0.0434	0.0519	44.2189
		✓	0.0461	0.0540	44.3755
			0.0001	0.0002	0.2506

Removing either seed construction or diversity shaping leads to a clear performance drop. Without diversity shaping, RankIC decreases from 0.0586 to 0.0505, suggesting that the Miner LLM tends to over-exploit similar factor patterns when diversity and complementarity signals are absent. Without seed construction, RankIC decreases to 0.0519, indicating that high-quality seed factors provide useful starting points for RFT and help guide the search toward promising regions of the factor

space. These results show that both seed initialization and diversity-aware reward shaping contribute to effective factor discovery.

Interestingly, the variant without both Seed and Div but still using the Factor DSL achieves a RankIC of 0.0540, which is lower than the full model but remains competitive. This suggests that the Factor DSL itself provides an important inductive bias by restricting the search space to executable and financially meaningful factor expressions. However, adding seed construction and diversity-aware shaping on top of the DSL further improves both factor quality and stability.

The last row removes the Factor DSL and lets the model generate unconstrained factor code. Its performance collapses to a RankIC of only 0.0002 and an ICIR of 0.2506. This demonstrates that unconstrained code generation is unreliable for RFT-based factor discovery: generated programs are more likely to be invalid, unstable, or poorly aligned with the evaluation objective. In contrast, the Factor DSL provides a structured action space that makes reinforcement fine-tuning feasible and sample-efficient.

Overall, the ablation study confirms that the strongest performance comes from the combination of all three components. The Factor DSL ensures executable and well-formed factor generation, seed construction provides high-quality starting points, and diversity-aware reward shaping prevents repetitive exploitation while encouraging complementary discoveries.

F. Hyperparameter Sensitivity

Figure 3 studies the sensitivity of factor fusion performance to two important hyperparameters on Benchmark B: the number of selected factors k and the correlation threshold used during factor selection. Since Benchmark B uses RankIC as the primary metric, we report the fused out-of-sample RankIC under different hyperparameter settings.

Figure 3(a) shows that QUANTEVOLVER consistently outperforms all baselines across different top- k settings. Its RankIC remains around 0.050 even when only the top three to five factors are selected, and steadily improves as more high-quality factors are included. The performance increases from 0.0498 at $k = 5$ to 0.053 at $k = 10$, 0.0562 at $k = 20$, and reaches its best value of 0.0587 around $k = 25$. The result remains nearly unchanged at $k = 30$ with a RankIC of 0.0586, and only slightly decreases when k is further enlarged. This trend indicates that QUANTEVOLVER produces a rich and

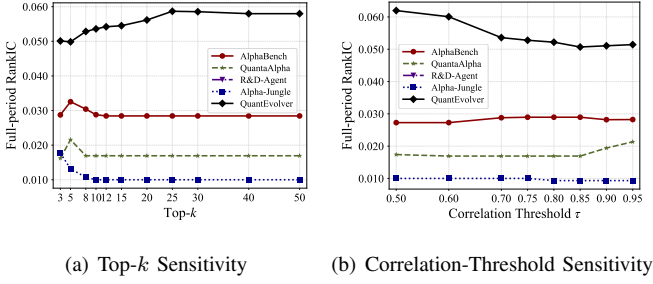


Fig. 3. Hyperparameter Sensitivity Experiment on Dataset B

stable factor pool: adding more selected factors improves the fused signal up to a moderate scale, while the performance remains robust when the selected set becomes larger.

Figure 3(b) examines the effect of the correlation threshold used during factor selection. A lower threshold imposes a stricter decorrelation constraint among selected factors. QUANTEVOLVER achieves the strongest performance under stricter thresholds, reaching a RankIC of 0.0620 at threshold 0.50 and 0.0601 at threshold 0.60. As the threshold becomes looser, the RankIC decreases to around 0.052–0.054, suggesting that controlling redundancy among selected factors is important for obtaining a stronger fused signal. This result indicates that the mined factor pool contains many individually predictive candidates, but effective fusion still benefits from explicitly preserving complementary information.

Overall, the sensitivity analysis shows that QUANTEVOLVER is robust to fusion hyperparameters. It maintains a clear advantage over baseline methods across a wide range of top- k values and correlation thresholds, while the best configurations favor a moderately sized factor set with relatively strict decorrelation. This further supports that QUANTEVOLVER discovers not only isolated strong factors, but also a diverse and complementary factor pool suitable for multi-factor fusion.

G. Mining Process Analysis

Figure 4 provides a closer look at the mining dynamics of QUANTEVOLVER and its ablated variants on Dataset B. In Figure 4(a), the full QUANTEVOLVER design exhibits a more stable and consistently improving reward trajectory over search progress, indicating that the combination of seed initialization and diversity-aware exploration helps the search process move toward more promising regions of the factor space. Removing either component leads to a weaker or less stable optimization trend, while removing both causes the mining process to become substantially less effective.

Figure 4(b) further examines the quality of high-scoring discovered factors. The full method maintains the strongest rolling top-10% score over most of the search process, showing that QUANTEVOLVER not only improves the average optimization trajectory but also discovers stronger high-quality candidates more reliably. In contrast, the ablated variants either improve more slowly or plateau at lower quality levels. These results suggest that seed-based initialization and diversity-aware search are both important for sustaining effective factor discovery throughout the mining process.

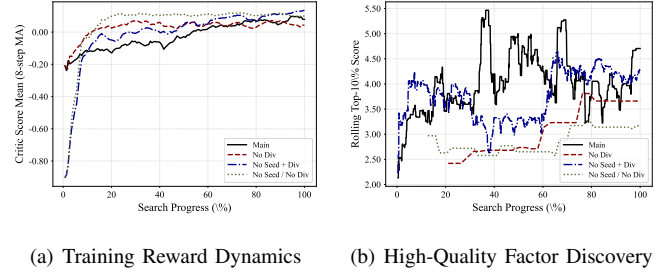


Fig. 4. Mining process analysis on Dataset B.

H. Profitability Case Study

Figure 5 presents a profitability case study on Benchmark B by converting discovered cross-sectional factors into a simple long-short portfolio. At each rebalancing timestamp, assets are ranked by the factor signal, and the portfolio takes long positions in the top-ranked assets and short positions in the bottom-ranked assets. This experiment is not intended to be a fully optimized trading system; instead, it serves as a practical sanity check on whether the higher RankIC achieved by QUANTEVOLVER can translate into economically meaningful returns.

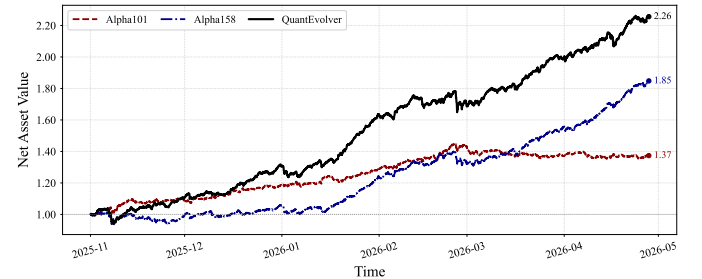


Fig. 5. Cumulative Return Comparison on Dataset B

The results show that the QUANTEVOLVER-based portfolio produces the strongest cumulative return among all compared factor sets. Over the evaluation period from November 2025 to April 2026, the net asset value of QUANTEVOLVER increases from 1.00 to about 2.26, corresponding to a cumulative return of approximately 125.6%. In comparison, Alpha158 reaches a final net asset value of about 1.85, while Alpha101 reaches about 1.37. Thus, QUANTEVOLVER not only improves the statistical ranking metric reported in previous sections, but also leads to a substantially stronger long-short return curve.

The return trajectory also indicates that QUANTEVOLVER maintains a persistent advantage over the classical alpha sets. Although all methods experience short-term fluctuations and drawdowns, the QUANTEVOLVER curve begins to separate from the baselines after the early evaluation period and continues to compound more rapidly. This suggests that the mined factors capture cross-sectional return signals that remain effective over time rather than producing only isolated high-IC observations.

Overall, this case study provides additional evidence that the factor pool discovered by QUANTEVOLVER is not merely statistically predictive, but also economically useful under a

simple portfolio construction protocol. The stronger cumulative return supports the practical relevance of policy-update-driven factor discovery for high-frequency cross-sectional alpha mining.

V. RELATED WORK

A. LLM-Based Quantitative Finance

With the rapid development of large language models (LLMs), increasing efforts have explored their applications in quantitative finance. From the perspective of financial tasks, existing work can be broadly divided into five categories: data analysis, investment research, trading, investment management, and risk management [25].

Data analysis focuses on using LLMs to process heterogeneous financial information, such as earnings-call transcripts, annual reports, financial news, SEC filings, and social media posts. Representative tasks include financial text summarization, named entity recognition, relation extraction, event classification, and sentiment analysis [26]–[28]. Domain-specific financial LLMs, such as FinBERT, BloombergGPT, FinGPT, FinMA, InvestLM, FinTral, and ICE-Intern, further improve financial language understanding by adapting general language models to financial corpora or instruction data [29]–[35].

Investment research aims to support market analysis, asset evaluation, forecasting, and investment hypothesis generation. Existing work studies LLMs for event classification, financial sentiment analysis, time-series forecasting, and investment reasoning over market data and textual information [36]–[40]. More closely related to this paper, recent studies apply LLMs to alpha factor mining, where LLMs are used as factor miners, evaluators, interactive assistants, or search controllers to generate and refine formulaic alpha expressions [12]–[19].

Trading focuses on using LLMs to support strategy execution and trading decision-making. In this setting, LLMs may interpret market states, summarize relevant signals, reason over trading opportunities, or generate executable trading decisions based on historical prices, news, reports, and sentiment information [41]–[44]. Other works further explore memory-augmented or agentic trading systems, where LLM agents coordinate market observation, reflection, and decision-making over time [45]–[48].

Investment management studies how LLMs can assist portfolio-level reasoning, asset allocation, financial question answering, and investment recommendation. Representative benchmarks and models focus on answering complex financial questions over reports, tables, and market information, requiring numerical reasoning, multi-hop reasoning, and risk-return trade-off analysis [49]–[51]. These studies show the potential of LLMs to support portfolio construction and investment decision workflows, but also reveal challenges in long-context reasoning, numerical calculation, and adaptation to dynamic financial data.

Risk management applies LLMs to tasks such as fraud detection, default risk prediction, compliance analysis, and financial risk monitoring. Existing studies and benchmarks examine how LLMs can identify abnormal transactions, assess borrower risk, and support regulatory or compliance-oriented

analysis [52]–[54]. These applications require models to handle imbalanced data, evolving risk patterns, and interpretable financial evidence.

Overall, LLM-based quantitative finance has evolved from financial information processing toward investment research, trading support, portfolio reasoning, and risk monitoring. This paper belongs to the investment research direction, with a specific focus on LLM-based alpha factor discovery.

B. Alpha Factor Discovery

Alpha factor discovery aims to automatically identify predictive signals from historical market data. Existing methods can be broadly categorized according to how they explore the factor expression space.

Early automated methods mainly rely on rule-based symbolic search. These methods represent alpha factors as symbolic expressions and search over the expression space using genetic programming, symbolic regression, evolutionary search, operator-template enumeration, or grammar-based constraints [1]–[6]. By explicitly defining variables, operators, and composition rules, symbolic-search methods can generate executable and interpretable factor expressions. However, their exploration process is often constrained by hand-crafted mutation rules, crossover strategies, search heuristics, or syntactic templates, making it difficult to scale efficiently to large and semantically rich factor spaces.

Another line of work introduces learning-guided factor generation. Instead of relying purely on heuristic symbolic search, these methods use trainable models, reinforcement learning policies, or neural generators to guide the construction, ranking, and selection of alpha expressions [5], [7]–[11]. For example, reinforcement-learning-based methods formulate factor discovery as a sequential expression-construction problem, where an agent selects variables and operators according to empirical rewards. Neural and generative methods further learn factor-generation patterns from historical evaluations or previously discovered factors. These approaches improve search adaptivity, but they usually require predefined action spaces, expression templates, or task-specific training objectives, and their generation ability is still limited by the learned search policy rather than open-ended semantic composition.

Recently, LLM-based methods have provided a new paradigm for alpha factor discovery. Leveraging their symbolic reasoning, code-like generation, and instruction-following capabilities, LLMs can generate and refine factor expressions from alpha libraries, market contexts, user instructions, and empirical feedback [12]–[19]. Existing LLM-based frameworks have explored interactive factor mining, factor evaluation and refinement, search-augmented generation, multi-agent workflows, and evolutionary exploration. These methods improve the flexibility of alpha mining, but most of them still use LLMs as prompt-driven miners, evaluators, or agents.

Our work is positioned at the intersection of learning-guided factor generation and LLM-based alpha mining. Unlike rule-based symbolic search, QUANTEVOLVER leverages a language-model policy to generate DSL factor expressions. Unlike existing prompt-driven LLM-based methods, it uses

reinforcement fine-tuning as a search-distribution optimization mechanism during factor mining. The goal is not merely to obtain a deployable factor-mining agent, but to collect, validate, and select a high-quality mined factor set throughout the training process.

VI. CONCLUSION

In this paper, we study LLM-based alpha factor discovery and identify two key limitations of existing prompt-level optimization methods: context explosion and search stagnation. To address these challenges, we propose QUANTEVOLVER, a self-evolving alpha factor discovery framework based on reinforcement fine-tuning. Instead of accumulating historical candidates and feedback in the prompt, QUANTEVOLVER converts executable quantitative evaluation into policy updates, enabling a Miner LLM to internalize optimization experience through parameter learning and progressively improve its factor generation policy. Experiments on three realistic market benchmarks demonstrate that QUANTEVOLVER consistently discovers stronger and more robust alpha factors than representative LLM-based baselines.

REFERENCES

- [1] T. Zhang, Y. Li, Y. Jin, and J. Li, "Autoalpha: an efficient hierarchical evolutionary algorithm for mining alpha factors in quantitative investment," *arXiv preprint arXiv:2002.08245*, 2020.
- [2] W. Ren, Y. Qin, and Y. Li, "Alpha mining and enhancing via warm start genetic programming for quantitative investment," *arXiv preprint arXiv:2412.00896*, 2024.
- [3] Z. Kakushadze, "101 formulaic alphas," *Wilmott*, vol. 2016, no. 84, pp. 72–81, 2016.
- [4] I. Arnaldo, K. Krawiec, and U.-M. O'Reilly, "Multiple regression genetic programming," in *Proceedings of the 2014 annual conference on genetic and evolutionary computation*, 2014, pp. 879–886.
- [5] H. Yang, D. Hao, Z. Wang, Q. Shi, and X. Li, "Alpha discovery via grammar-guided learning and search," *arXiv preprint arXiv:2601.22119*, 2026.
- [6] T. Ren, R. Zhou, J. Jiang, J. Liang, Q. Wang, and Y. Peng, "Riskminer: Discovering formulaic alphas via risk seeking monte carlo tree search," in *Proceedings of the 5th ACM International Conference on AI in Finance*, 2024, pp. 752–760.
- [7] S. Yu, H. Xue, X. Ao, F. Pan, J. He, D. Tu, and Q. He, "Generating synergistic formulaic alpha collections via reinforcement learning," in *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining*, 2023, pp. 5476–5486.
- [8] F. Xu, Y. Yin, X. Zhang, T. Liu, S. Jiang, and Z. Zhang, "Alpha2: Discovering logical formulaic alphas using deep reinforcement learning," *arXiv preprint arXiv:2406.16505*, 2024.
- [9] Z. Zhu and K. Zhu, "Alphaqcm: Alpha discovery in finance with distributional reinforcement learning," in *Forty-second International Conference on Machine Learning*, 2025.
- [10] H. Shi, W. Song, X. Zhang, J. Shi, C. Luo, X. Ao, H. Arian, and L. A. Seco, "Alphaforge: A framework to mine and dynamically combine formulaic alpha factors," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 39, no. 12, 2025, pp. 12 524–12 532.
- [11] B. Chen, H. Ding, N. Shen, J. Huang, T. Guo, L. Liu, and M. Zhang, "Alphasage: Structure-aware alpha mining via gflownets for robust exploration," *arXiv preprint arXiv:2509.25055*, 2025.
- [12] S. Wang, H. Yuan, L. Zhou, L. Ni, H. Y. Shum, and J. Guo, "Alpha-gpt: Human-ai interactive alpha mining for quantitative investment," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2025, pp. 196–206.
- [13] Z. Li, R. Song, C. Sun, W. Xu, Z. Yu, and J.-R. Wen, "Can large language models mine interpretable financial factors more effectively? a neural-symbolic factor mining agent model," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 3891–3902.
- [14] S. Wang, H. Yuan, L. M. Ni, and J. Guo, "Quantagent: Seeking holy grail in trading by self-improving large language model," *arXiv preprint arXiv:2402.03755*, 2024.
- [15] H. Luo, H. T. Ko, J. Chen, D. Sun, Y. Zhang, and C. Liu, "Al-phabench: Benchmarking large language models in formulaic alpha factor mining," in *The Fourteenth International Conference on Learning Representations*.
- [16] Z. Tang, Z. Chen, J. Yang, J. Mai, Y. Zheng, K. Wang, J. Chen, and L. Lin, "Alphaagent: Llm-driven alpha mining with regularized exploration to counteract alpha decay," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2025, pp. 2813–2822.
- [17] Y. Shi, Y. Duan, and J. Li, "Navigating the alpha jungle: An llm-powered mcts framework for formulaic alpha factor mining," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 2, 2026, pp. 997–1005.
- [18] Y. Li, X. Yang, X. Yang, X. Wang, W. Liu, and J. Bian, "R&d-agent-quant: a multi-agent framework for data-centric factors and model joint optimization," *Advances in Neural Information Processing Systems*, vol. 38, 2026.
- [19] J. Han, S. Zhang, W. Li, Z. Yang, Y. Dong, T. Hu, J. Yuan, X. Yu, Y. Zhu, F. Lou et al., "Quantaalpha: An evolutionary framework for llm-driven alpha mining," *arXiv preprint arXiv:2602.07085*, 2026.
- [20] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, "Deep reinforcement learning from human preferences," *Advances in neural information processing systems*, vol. 30, 2017.
- [21] D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, and G. Irving, "Fine-tuning language models from human preferences," *arXiv preprint arXiv:1909.08593*, 2019.
- [22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.
- [23] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," *Advances in Neural Information Processing Systems*, vol. 36, pp. 53 728–53 741, 2023.
- [24] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu et al., "Deepseekmath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.
- [25] Y. Dong, F. Wu, K. Zhang, Y. Dai, S. Zhang, W. Ye, S. Chen, and Z.-Q. Cheng, "Large language model agents in finance: A survey bridging research, practice, and real-world deployment," *Findings of the Association for Computational Linguistics: EMNLP*, vol. 2025, pp. 17 889–17 907, 2025.
- [26] R. Mukherjee, A. Bohra, A. Banerjee, S. Sharma, M. Hegde, A. Shaikh, S. Shrivastava, K. Dasgupta, N. Ganguly, S. Ghosh et al., "Ectsum: A new benchmark dataset for bullet point summarization of long earnings call transcripts," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 2022, pp. 10 893–10 906.
- [27] H. Li, Q. Peng, X. Mou, Y. Wang, Z. Zeng, and M. F. Bashir, "Abstractive financial news summarization via transformer-bilstm encoder and graph attention-based decoder," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 3190–3205, 2023.
- [28] S. Sharma, T. Nayak, A. Bose, A. K. Meena, K. Dasgupta, N. Ganguly, and P. Goyal, "Finred: A dataset for relation extraction in financial domain," in *Companion Proceedings of the Web Conference 2022*, 2022, pp. 595–597.
- [29] Z. Liu, D. Huang, K. Huang, Z. Li, and J. Zhao, "Finbert: A pre-trained financial language representation model for financial text mining," in *Proceedings of the twenty-ninth international conference on international joint conferences on artificial intelligence*, 2021, pp. 4513–4519.
- [30] S. Wu, O. Irsoy, S. Lu, V. Dabravolski, M. Dredze, S. Gehrmann, P. Kambadur, D. Rosenberg, and G. Mann, "Bloomberggpt: A large language model for finance," *arXiv preprint arXiv:2303.17564*, 2023.
- [31] H. Yang, X.-Y. Liu, and C. D. Wang, "Fingpt: Open-source financial large language models," *arXiv preprint arXiv:2306.06031*, 2023.
- [32] Q. Xie, W. Han, X. Zhang, Y. Lai, M. Peng, A. Lopez-Lira, and J. Huang, "Pixiu: a large language model, instruction data and evaluation benchmark for finance," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, 2023, pp. 33 469–33 484.
- [33] Y. Yang, Y. Tang, and K. Y. Tam, "Investlm: A large language model for investment using financial domain instruction tuning," *arXiv preprint arXiv:2309.13064*, 2023.
- [34] G. Bhatia, H. Cavusoglu, M. Abdul-Mageed et al., "Fintral: A family of gpt-4 level multimodal financial large language models," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 13 064–13 087.

- [35] G. Hu, K. Qin, C. Yuan, M. Peng, A. Lopez-Lira, B. Wang, S. Ananiadou, J. Huang, and Q. Xie, “No language is an island: Unifying chinese and english in financial large language models, instruction data, and benchmarks,” *arXiv preprint arXiv:2403.06249*, 2024.
- [36] J. Lee, H. L. Youn, N. Stevens, J. Poon, and S. C. Han, “Fednlp: an interpretable nlp system to decode federal reserve communications,” in *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*, 2021, pp. 2560–2564.
- [37] A. Shah, S. Paturi, and S. Chava, “Trillion dollar words: A new financial dataset, task & market analysis,” in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, pp. 6664–6679.
- [38] A. Sinha and T. Khandait, “Impact of news on the commodity market: Dataset and results,” in *Future of Information and Communication Conference*. Springer, 2021, pp. 589–601.
- [39] X. Yu, Z. Chen, and Y. Lu, “Harnessing llms for temporal data—a study on explainable financial time series forecasting,” in *Proceedings of the 2023 conference on empirical methods in natural language processing: industry track*, 2023, pp. 739–753.
- [40] Y. Hu, Y. Li, P. Liu, Y. Zhu, N. Li, T. Dai, S.-t. Xia, D. Cheng, and C. Jiang, “Fintsb: A comprehensive and practical benchmark for financial time series forecasting,” *arXiv preprint arXiv:2502.18834*, 2025.
- [41] U. Gupta, “Gpt-investar: Enhancing stock investment strategies through annual report analysis with large language models,” *arXiv preprint arXiv:2309.03079*, 2023.
- [42] Q. Xie, W. Han, Z. Chen, R. Xiang, X. Zhang, Y. He, M. Xiao, D. Li, Y. Dai, D. Feng et al., “Finben: A holistic financial benchmark for large language models,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 95 716–95 743, 2024.
- [43] H. Li, Y. Cao, Y. Yu, S. R. Javaji, Z. Deng, Y. He, Y. Jiang, Z. Zhu, K. Subbalakshmi, J. Huang et al., “Investorbench: A benchmark for financial decision-making tasks with llm-based agent,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 2509–2525.
- [44] Y. Lu, Y. Hu, H. Foroosh, W. Jin, and F. Liu, “Strux: An llm for decision-making with structured explanations,” in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)*, 2025, pp. 131–141.
- [45] Y. Yu, H. Li, Z. Chen, Y. Jiang, Y. Li, J. W. Suchow, D. Zhang, and K. Khashanah, “Finmem: A performance-enhanced llm trading agent with layered memory and character design,” *IEEE Transactions on Big Data*, 2025.
- [46] J. Li, Y. Bian, G. Wang, Y. Lei, D. Cheng, Z. Ding, and C. Jiang, “Cfpgpt: Chinese financial assistant with large language model,” *arXiv preprint arXiv:2309.10654*, 2023.
- [47] C. Zhang, X. Liu, Z. Zhang, M. Jin, L. Li, Z. Wang, W. Hua, D. Shu, S. Zhu, X. Jin et al., “When ai meets finance (stockagent): Large language model-based stock trading in simulated real-world environments,” *arXiv preprint arXiv:2407.18957*, 2024.
- [48] Y. Xiao, E. Sun, D. Luo, and W. Wang, “Tradingagents: Multi-agents llm financial trading framework,” in *The First MARW: Multi-Agent AI in the Real World Workshop at AAAI 2025*.
- [49] Z. Chen, S. Li, C. Smiley, Z. Ma, S. Shah, and W. Y. Wang, “Convfinqa: Exploring the chain of numerical reasoning in conversational finance question answering,” in *Proceedings of the 2022 conference on empirical methods in natural language processing*, 2022, pp. 6279–6292.
- [50] C. Choi, J. Kwon, J. Ha, H. Choi, C. Kim, Y. Lee, J.-y. Sohn, and A. Lopez-Lira, “Finder: Financial dataset for question answering and evaluating retrieval-augmented generation,” in *Proceedings of the 6th ACM International Conference on AI in Finance*, 2025, pp. 638–646.
- [51] X. Li, Z. Li, C. Shi, Y. Xu, Q. Du, M. Tan, and J. Huang, “Al-phafin: Benchmarking financial analysis with retrieval-augmented stock-chain framework,” in *Proceedings of the 2024 joint international conference on computational linguistics, language resources and evaluation (LREC-COLING 2024)*, 2024, pp. 773–783.
- [52] N. Balasubramanian, Y. Ye, and M. Xu, “Substituting human decision-making with machine learning: Implications for organizational learning,” *Academy of Management Review*, vol. 47, no. 3, pp. 448–465, 2022.
- [53] Y. Yin, Y. Yang, J. Yang, and Q. Liu, “Finpt: Financial risk prediction with profile tuning on pretrained foundation models,” *arXiv preprint arXiv:2308.00065*, 2023.
- [54] D. Feng, Y. Dai, J. Huang, Y. Zhang, Q. Xie, W. Han, Z. Chen, A. Lopez-Lira, and H. Wang, “Empowering many, biasing a few: Generalist credit scoring through large language models,” *arXiv preprint arXiv:2310.00566*, 2023.